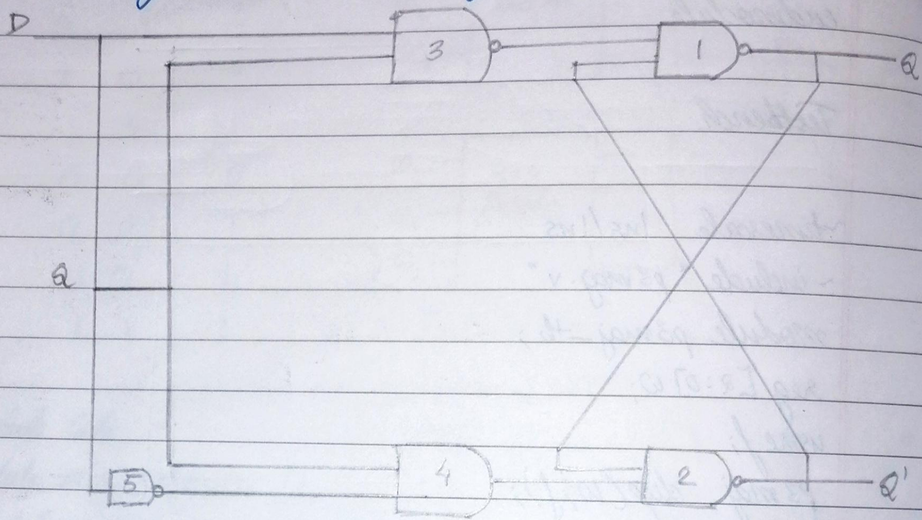
Lab No. 6
FlipFlops

1. Write behavioral Verilog code for a positive edge-triggered D FF with asynchronous active high reset.



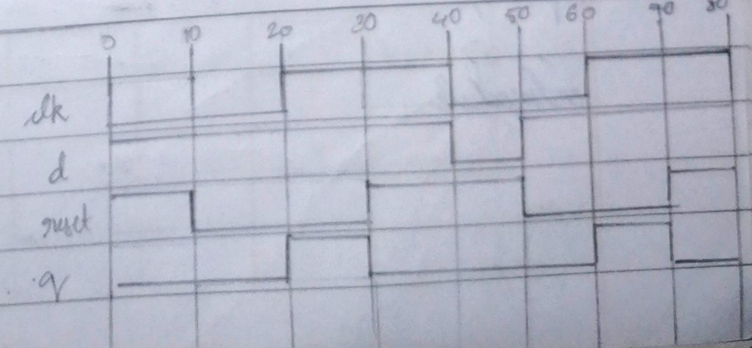
Q	D	Q(t+1)
0	0	0
0	1	1
1	0	0
1	1	1

Code

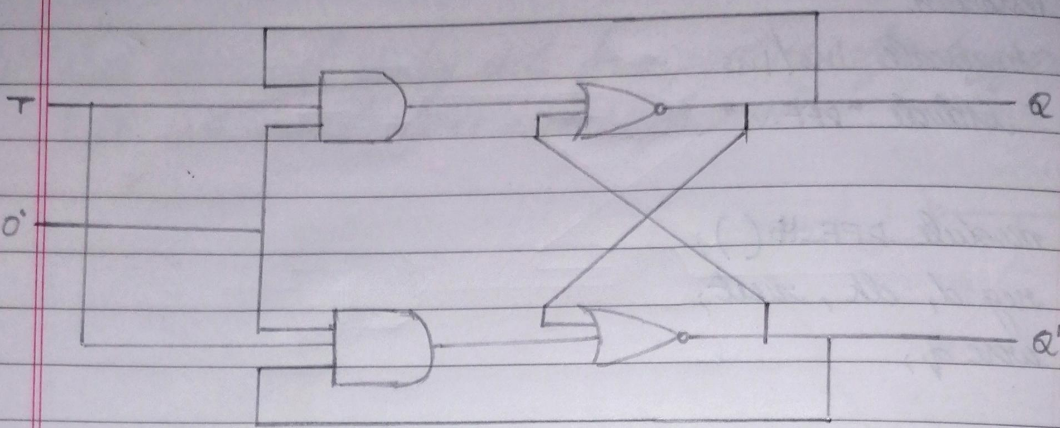
```
module DFF(d, clk, reset, q);
input d, clk, reset;
output reg q;

always @(posedge clk)
if (!reset)
q<=d;

always @(reset)
if(reset)
q<=0;
```

```
endmodule

Testbench
`timescale 1ns/1ns
`include "DFF.v"

module DFF_tb();
reg d, clk, reset;
wire q;

DFF DFFq (d, clk, reset, q);
always #20 clk = ~clk;
initial
begin
    $dumpfile("DFF_tb.vcd");
    $dumpvars(0, DFF_tb);
    clk = 0;
    #80 $finish;
    end
    initial
    begin
    d=1; reset=1; #10;
    d=1; reset=0; #20;
    d=1; reset=1; #10;
    d=0; reset=1; #10;
    d=1; reset=0; #20;
    d=0; reset=0; #10;
    $display("Test complete");
    end
endmodule
```

Waveform



Q. Write behavioral Verilog code for a negative edge triggered T FF with asynchronous active low reset.



a) Logic circuit

Q	T	Q(t+1)
0	0	0
0	1	1
1	0	1
1	1	0

$Q(t+1) = T'Q + TQ'$

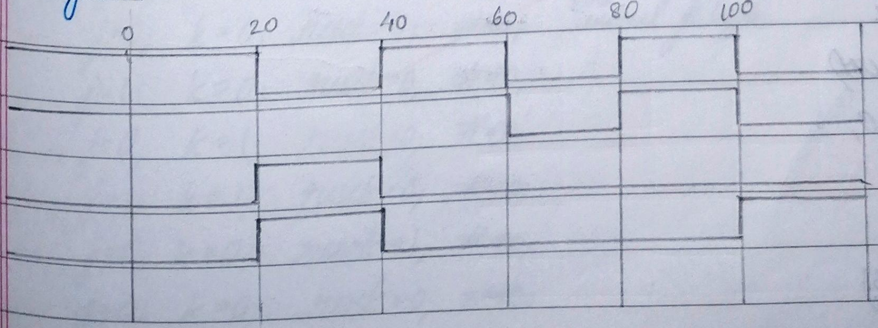
Code

```
module TFF (t, clk, reset, q);
input t, clk, reset;
output reg q;
always @(negedge clk)
if (reset)
q <= ~q;

always @(reset)
if (!reset)
q <=0;

endmodule
```

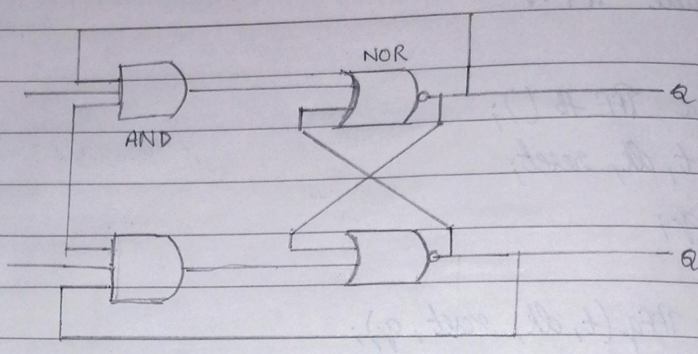
Testbench

```
`timescale 1ns/1ns
`include "TFF.v"

module TFF_tb ();
reg t, clk, reset;
wire q;

TFF TFFq (t, clk, reset, q);
always #20 clk = ~clk;
initial begin
        $dumpfile ("TFF_tb.vcd");
        $dumpvars (0, TFF_tb);
        clk = 1;
        #110 $finish;
        end
        initial begin
        t=1; reset=0; #20;
        t=1; reset=1; #20;
        t=1; reset=0; #20;
        t=0; reset=0; #20;
        t=1; reset=1; #20;
        t=0; reset=1; #20;
        $display ("Test Complete");
        end
endmodule
```

Waveform



3. Write behavioral Verilog code for a positive edge-triggered JK FF with synchronous active high reset.



Q	J	K	Q(t+1)
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

$$Q(t+1) = JQ' + KQ'$$

Code

```
module jkFF (j, k, clk, reset, q);
input j, k, clk, reset;
output reg q;

always @(posedge clk)
begin
if (reset)
q <=0;
else
begin
if (clk)
```

```
begin
if (!q)
begin
if (j)
q <= 1;
end
else
begin
if (k)
q <= 0;
end
end
end
end
endmodule
```

Testbench

```
`timescale 1ns/1ns
`include "jkFF.v"

module jkFF_tb();
reg j, k, clk, reset;
wire q;

jkFF jkFFq (j, k, clk, reset, q);
always #10 clk = ~clk;
initial begin
    j=0; k=0; reset=1; #20;
    j=0; k=0; reset=0; #20;
    j=1; k=0; reset=0; #20;
    j=0; k=1; reset=0; #20;
    j=1; k=1; reset=0; #20;
    j=0; k=0; reset=1; #20;
    j=0; k=0; rest=0; #20;
```

```
$dumpfile ("jkFF_tb.vcd");
$dumpvars (0
clk = 1;
#120 $finish;
end
initial begin
```

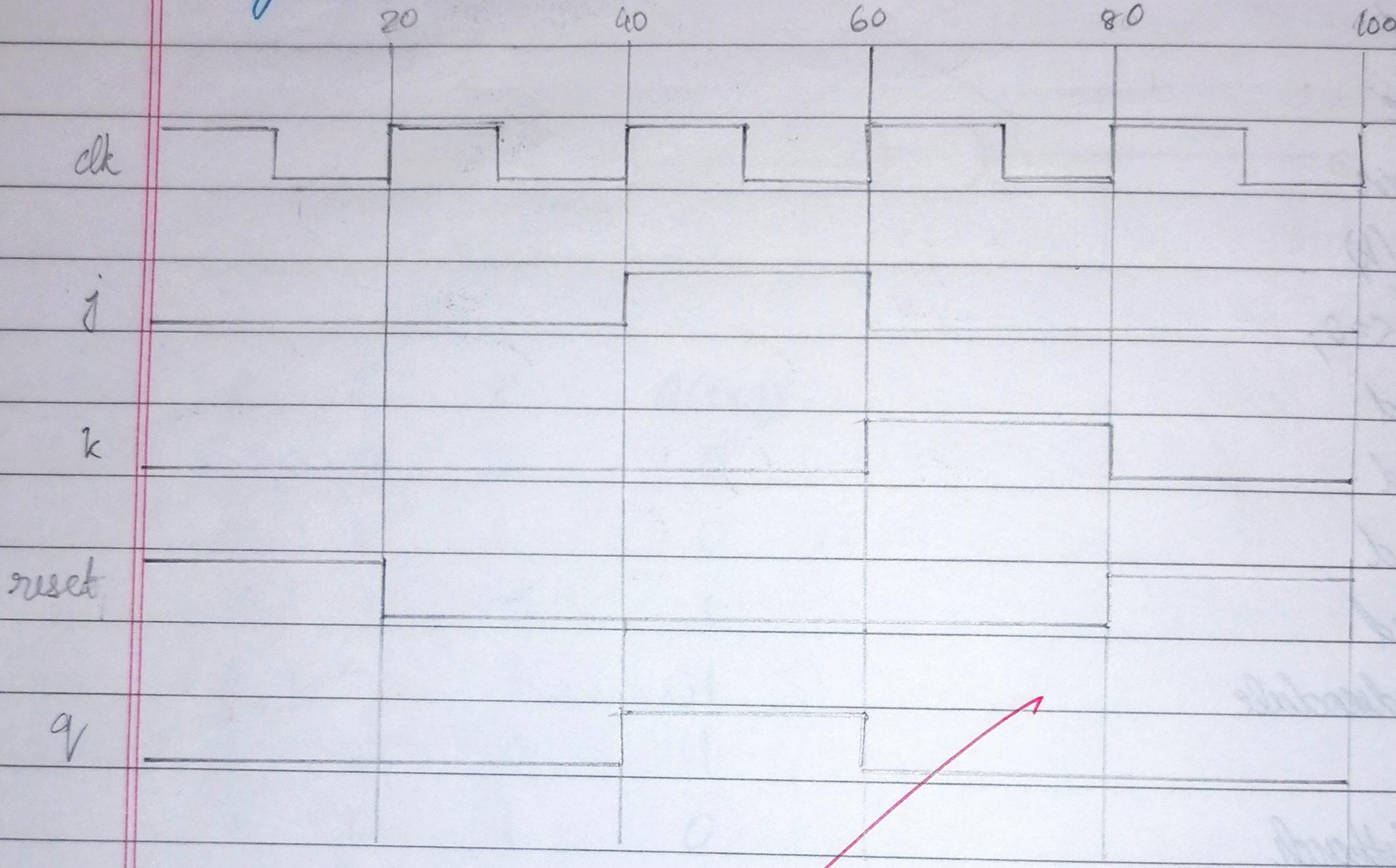
Date 14/10/23
Page

~~end~~ $display("Test complete");

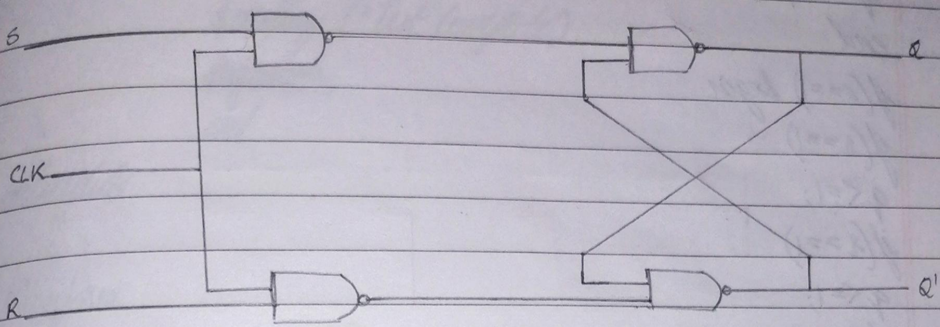
~~endmodule~~ end

endmodule

Waveform



4. Write behavioral Verilog code for a negative edge-triggered SR FF with synchronous active low reset.



Q	S	R	Q(t+1)
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	intermediate
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	intermediate

S	R	Q(t+1)
0	0	Q
0	1	0
1	0	1
1	1	?

$$Q(t+1) = S + R'Q$$

$$SR = 0$$

```
module srff(s, r, clk, reset, q);
input s, r, clk, reset;
output reg q;
always @ (negedge clk)
begin
if(!reset)
q <= 1'b0;
else
begin
if(s==0) begin
if(r==0)
```

```
q<=q;
if(r==0)
q<=0;
end
if(s==1) begin
if(r==1)
q<=1;
if(r==1)
q<=1;
if(r==1)
q<=1'bx;
end
end
end
endmodule
```

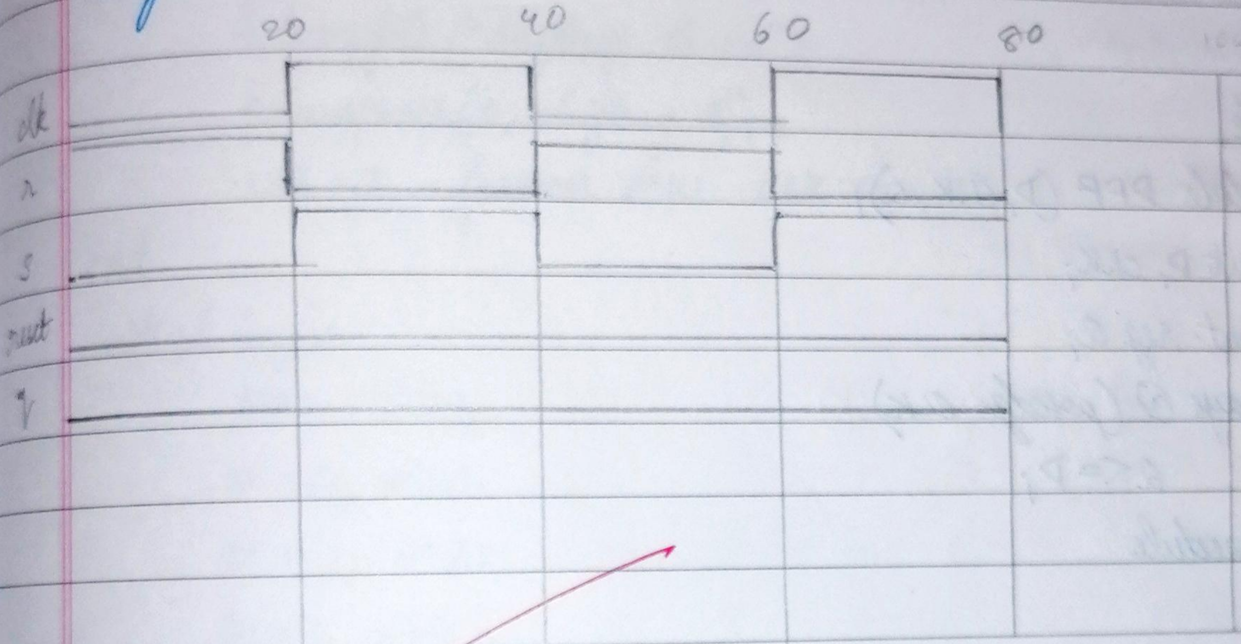
Testbench

```
`timescale 1ns/1ns
`include "SRFF.v"

module SRFF_tb();
reg s, r, clk, reset;
wire q;

SRFF test(s, r, clk, reset, q);
initial begin
~~SRFF test(s, r, clk, res~~ $dumpfile("SRFF_tb.vcd");
$dumpvars(0, SRFF_tb);
clk = ~~1'b0~~ 1'b0;
forever #10 clk=~clk;
end
initial begin
s=1; r=0; reset=1; #20;
s=0; r=0; reset=0; #20;
```

Date 14/10/23

```
s=0; r=1; reset=1; #20;
s=0; r=0; reset=1; #20;
s=1; r=1; reset=1; #20;
$display ("Test Complete");
~~$finish~~;
end
endmodule
```

Waveform



14/10/23